

Desain UTS Kelompok 6 T2 k9

Paket desain UTS — Kelompok 6 / T2 POS UMKM / k9

Koreksi lisan terbaru dosen yang dicatat kelompok menetapkan tiga dimensi domain (`dim_product`, `dim_outlet`, `dim_customer`) ditambah `dim_date`, jadi empat tabel dimensi total. Outlet A, Januari–Juni 2025 sudah memenuhi scope cut. D7 tersedia sebagai `docs/D7_scope_cut.md` di repo dan tidak dilampirkan pada PDF; target 2 dari 3 metrik, 3 tile, dan deadline 30 September 2026 telah disepakati kelompok pada 29 September 2026. Tidak ada loader, test runner, warehouse, atau dashboard yang dijalankan.

1. Brief, star schema, dan DDL

Status: jumlah dimensi dan pemotongan scope sudah dikonfirmasi dosen; target jumlah metrik/tile dan deadline D7 telah disepakati kelompok pada 29 September 2026. **Batas:** Outlet A (OUT-A) dan transaksi tanggal lokal WIB 1 Januari–30 Juni 2025. Sumber penugasan adalah sandbox/kelompok_topik.md; folder data/raw/t2_umkm/ adalah dataset yang diberikan. Scope –20% tidak menghapus tujuh deliverable UTS.

Tujuan dan keputusan

Pengguna yang diusulkan ialah **Manajer Outlet A**. Keputusan yang didukung adalah bulan mana perlu ditinjau karena perubahan nilai penjualan item PAID, produk mana memberi kontribusi terbesar, dan kapan proporsi transaksi berstatus REFUND naik. Hasilnya membantu prioritas pemeriksaan operasional; data ini tidak cukup untuk menyatakan laba atau nilai refund bersih.

Pertanyaan bisnis:

- 1. Berapa nilai item PAID valid dan jumlah transaksi PAID per bulan Januari–Juni 2025?
- 2. Produk apa yang kontribusi nilai item PAID validnya tertinggi setiap bulan?
- 3. Berapa jumlah dan rasio transaksi berstatus REFUND per bulan?

Jawaban yang diharapkan dari profiling sumber dengan kebijakan kualitas dalam desain:

Query	Kolom keluaran	Satu kalimat jawaban yang diharapkan
q01_tren_bulanan.sql	bulan, nilai_paid_rp, transaksi_paid	Mei 2025 tertinggi dengan nilai item PAID valid Rp33.051.000 dari 413 transaksi.
q02.sql	bulan, product_id, nama_produk_tercatat, nilai_paid_rp, peringkat	PRD-0003 berada di peringkat pertama pada Februari, Mei, dan Juni 2025.
q03.sql	bulan, transaksi_refund, transaksi_tercatat, rasio_refund_persen	Juni 2025 mencatat 8 transaksi REFUND dari 420 transaksi pada fact, sekitar 1,90%.

Angka di atas berasal dari docs/profile_t2_k9_slice.md, dihitung baca-saja pada CSV setelah aturan deduplikasi/karantina yang dirancang. **Query warehouse belum dijalankan.** Setelah implementasi diperbolehkan, hasil fact harus direkonsiliasi dengan angka ini dan pengecualian sumber.

Sumber dan batas data

Sumber	Key dan peran	Pemakaian
transactions.csv	transaction_id; outlet_id, customer_id, tanggal_waktu, status, total_bayar	Menetapkan outlet, waktu WIB, status, dan degenerate key.
transaction_items.csv	item_id, transaction_id, product_id	Grain dan nilai item: qty, harga transaksi, diskon.

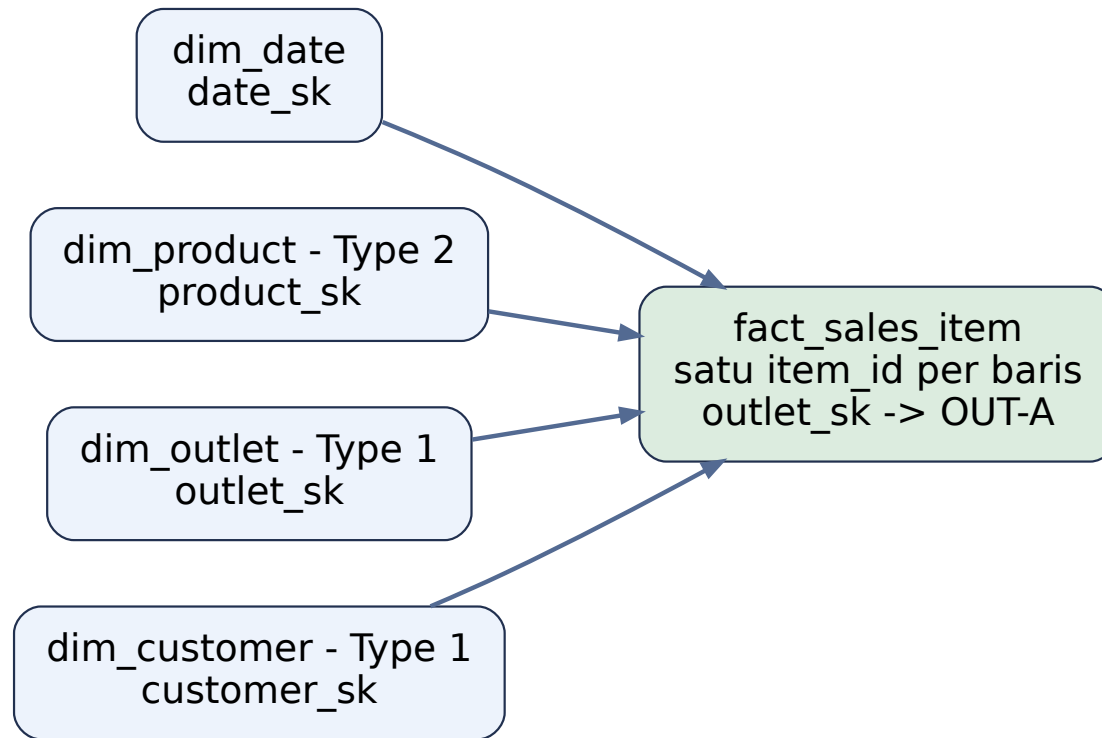
Sumber	Key dan peran	Pemakaian
products.csv	product_id	Nama/kategori dan referensi produk; harga master tidak menggantikan harga item.
outlets.csv	outlet_id	Memastikan OUT-A valid dan unik pada master; sumber dim_outlet Type 1.
customers.csv	customer_id	Atribut pelanggan; ID kosong menjadi pembeli anonim.

Satu folder sumber T2 berisi semua outlet/periode. Profiling bawaan `pipeline.profile --topic t2 --slice k9` masih menghitung seluruh CSV; test dan angka desain memakai query tambahan yang benar-benar memfilter slice. String waktu bertanda Z diperlakukan UTC lalu dikonversi ke WIB. String tanpa zona diasumsikan WIB. Data yang tidak dapat diparse tidak dimasukkan secara spekulatif ke periode.

Temuan slice: 2.578 baris header untuk 2.543 ID transaksi, 5.074 baris item terkait untuk 5.032 ID item, 34 kelompok header duplikat, empat ID header konflik, 42 kelompok item duplikat identik, 92 item qty negatif PAID (ditambah 2 pada VOID), dan 21 header tanpa item. Enam angka test yang tepat serta SQL buktinya berada di `docs/profile_t2_k9_slice.md`.

Grain dan star schema

Grain fact: tepat satu baris per `item_id` pada transaksi valid Outlet A dalam periode enam bulan, setelah item identik dide-duplikasi dan header konflik/PAID bermasalah dikarantina. `transaction_id` tetap di fact sebagai **degenerate dimension**; produk yang sama pada dua item transaksi tetap dua baris. Header konflik tidak dipilih acak. `total_bayar` header tidak diulang di fact karena akan menggandakan angka.



Star schema: empat dimensi menuju satu fact pada grain item_id.

DDL: sql/10_dim_date.sql (diberikan), sql/20_dim_product.sql, sql/20_dim_outlet.sql, sql/20_dim_customer.sql, dan sql/30_fact_sales_item.sql. Fact menyimpan empat FK dimensi, item_id, transaction_id, outlet_sk hasil lookup OUT-A dari header tervalidasi, waktu WIB, status, qty, harga item, diskon item, qty_paid, dan nilai_paid_rp.

Koreksi terbaru dosen: tiga tabel dimensi domain **ditambah** dim_date, sehingga total ada empat: dim_date, dim_product, dim_outlet, dan dim_customer. dim_outlet aktif walaupun slice hanya OUT-A; fact menyimpan outlet_sk. Tiga file 20_dim_*.sql kini sesuai struktur checker. Koreksi lisan dosen sudah dicatat kelompok; keputusan aktif dan konsekuensi desainnya diringkas di paragraf ini.

SCD dan aditivitas

dim_product adalah **SCD Type 2** dengan product_sk stabil per versi, valid_from, valid_to, is_current, dan lookup [valid_from, valid_to). Snapshot master tunggal tidak membuktikan harga/kategori masa lalu. Versi awal memakai tanggal sentinel "as known from snapshot" untuk lookup dan perubahan berikutnya disimpan sebagai versi baru ketika benar-benar teramati. Nilai faktual transaksi tetap dari harga item. dim_customer dan dim_outlet Type 1 karena koreksi atribut pelanggan/outlet pada scope ini tidak membutuhkan histori analitik; pelanggan kosong memakai anggota ANONIM.

dim_product	
product_sk 🔗	bigint
product_id	varchar NN
nama_produk	varchar NN
kategori	varchar
harga_referensi_rp	decimal(18,2)
harga_berlaku_dari_sumber	date
aktif	boolean
valid_from	date NN
valid_to	date NN
is_current	boolean NN

dim_customer	
customer_sk 🔗	bigint
customer_id	varchar NN
nama_pelanggan	varchar
kota	varchar
tanggal_daftar	date

fact_sales_item	
item_id 🔗	varchar
transaction_id	varchar NN
outlet_sk 🔗	bigint NN
date_sk 🔗	integer NN
product_sk 🔗	bigint NN
customer_sk 🔗	bigint NN
waktu_wib	timestamp NN
status_transaksi	varchar NN
qty_item	integer NN
harga_satuan_rp	decimal(18,2) NN
diskon_item_rp	decimal(18,2) NN
qty_paid	integer NN
nilai_paid_rp	decimal(18,2) NN

dim_outlet	
outlet_sk 🔗	bigint
outlet_id	varchar NN
nama_outlet	varchar NN
kota	varchar
tipe	varchar
aktif	boolean
dibuka_sejak	date

dim_date	
date_sk 🔗	integer
full_date	timestamp
tahun	integer
triwulan	integer
bulan	integer
nama_bulan	varchar
pekan_iso	integer
hari	integer
hari_ke	integer
nama_hari	varchar
akhir_pekan	boolean

Measure fact	Aditivitas	Aturan
nilai_paid_rp	Additive	SUM aman lintas tanggal, produk, outlet, dan pelanggan setelah satu item valid hanya satu baris; outlet slice tetap OUT-A; REFUND/VOID bernilai 0.
qty_paid	Additive secara aritmetika	SUM boleh untuk jumlah unit PAID; perbandingan lintas produk berbeda memerlukan kehati-hatian makna unit.
qty_item	Non-additive tanpa filter	REFUND memiliki tanda qty campuran; SUM semua status tidak berarti penjualan bersih.
harga_satuan_rp	Non-additive	Harga per unit; gunakan agregasi berbobot bila mencari harga rata-rata, jangan SUM.
diskon_item_rp	Non-additive lintas status	Diskon sumber disimpan untuk audit; untuk total diskon PAID, filter status dan validitas terlebih dahulu.
Jumlah transaksi (turunan query)	Non-additive antar-grup item	Hitung COUNT(DISTINCT transaction_id), bukan COUNT(*) fact.
Rasio refund (turunan query)	Non-additive	Hitung ulang pembilang/penyebut per potongan, jangan SUM rasio.

Asumsi finansial: diskon ditafsirkan nominal rupiah per baris item, sehingga $\text{nilai_paid_rp} = \text{qty} * \text{harga_satuan} - \text{diskon}$ pada PAID valid. Nilai yang ditemukan 0/1.000/2.000/5.000 mendukung asumsi, tetapi metadata satuan tidak tersedia; pemilik data harus mengonfirmasi sebelum pemakaian resmi. Status REFUND tidak dipakai untuk mengurangi metrik ini karena qty dan tautan ke transaksi asal tidak konsisten/tersedia. Ini **nilai item PAID tercatat**, bukan laba atau net setelah refund.

Alur load, test, dan metrik

pipeline/DESIGN_load.md menetapkan full replace staging setiap run, filter transaksi k9, deduplikasi identik, karantina konflik dan anomali PAID, MERGE dimensi, lalu MERGE fact_sales_item by item_id dengan pembersihan baris target k9 yang tidak lagi valid. Natural key, window, risiko penggandaan, penanganan SCD, dan idempotensi ditulis per tabel. Tidak ada loader yang dijalankan pada UTS.

Enam test topic: t2 dalam tests/test_definitions.yml memeriksa duplikat header (34), konflik header (4), duplikat item (42), qty negatif PAID (92), header tanpa item (21), dan orphan produk pada item slice (0). Lima test diperkirakan fail pada sumber, satu pass; setiap angka dan query di docs/profile_t2_k9_slice.md. Test runner tidak dijalankan.

Satu metrik UTS adalah **nilai item PAID valid Outlet A**. Dua belas field, SQL sql/50_metrics/nilai_item_paid.sql, cara gaming diskon diubah menjadi nol, dan guard perbandingan diskon/nilai terhadap staging ada di docs/kamus_metrik.md.

DDL dimensi dan fact

10_dim_date.sql

Sumber repo: sql/10_dim_date.sql

```
-- =====
-- 10_dim_date.sql – DIBERIKAN LENGKAP. Ini bukan checkpoint, ini alat.
-- Konformed dimension: SEMUA fact di warehouse ini menunjuk ke sini.
-- =====

CREATE OR REPLACE TABLE dim_date AS
SELECT CAST(strftime(d, '%Y%m%d') AS INTEGER) AS date_sk, -- kunci: YYYYMMDD, bukan urutan
       d AS full_date,
       CAST(year(d) AS INTEGER) AS tahun,
       CAST(quarter(d) AS INTEGER) AS triwulan,
       CAST(month(d) AS INTEGER) AS bulan,
       strftime(d, '%B') AS nama_bulan,
       CAST(week(d) AS INTEGER) AS pekan_iso,
       CAST(day(d) AS INTEGER) AS hari,
       CAST(dayofweek(d) AS INTEGER) AS hari_ke, -- 0 = Minggu
       strftime(d, '%A') AS nama_hari,
       CAST(dayofweek(d) IN (0, 6) AS BOOLEAN) AS akhir_pekan
FROM (SELECT unnest(generate_series(DATE '2024-01-01', DATE '2027-12-31', INTERVAL 1 DAY)) AS d);

-- Anggota Unknown: banyak pipeline gagal bukan karena datanya salah, tapi karena ada baris
-- yang tidak punya tanggal. Baris seperti itu tetap harus punya tempat.
INSERT INTO dim_date
SELECT -1, DATE '1900-01-01', 1900, 0, 0, 'TIDAK DIKETAHUI', 0, 0, -1, 'TIDAK DIKETAHUI', FALSE;
```

20_dim_product.sql

Sumber repo: [sql/20_dim_product.sql](#)

```
-- UTS K6/T2/k9. SCD Type 2: identitas produk tetap, atribut produk dapat berubah.
-- Grain: satu versi atribut untuk satu product_id; interval [valid_from, valid_to).
-- Seed awal berasal dari satu snapshot products.csv: atribut masa lalu tidak diketahui.
-- valid_from awal 1900-01-01 adalah penanda backfill "as known", bukan bukti harga historis.
-- Harga transaksi fact selalu memakai transaction_items.harga_satuan.
CREATE SEQUENCE IF NOT EXISTS seq_product_sk START 1;
CREATE TABLE IF NOT EXISTS dim_product (
    product_sk BIGINT PRIMARY KEY,
    product_id VARCHAR NOT NULL,
    nama_produk VARCHAR NOT NULL,
    kategori VARCHAR,
    harga_referensi_rp DECIMAL(18,2),
    harga_berlaku_dari_sumber DATE,
    aktif BOOLEAN,
    valid_from DATE NOT NULL,
    valid_to DATE NOT NULL,
    is_current BOOLEAN NOT NULL,
    CHECK (valid_from < valid_to)
);
-- Loader mengalokasikan nextval('seq_product_sk') sekali per versi baru; -1 untuk Unknown.
-- UNIQUE(product_id, valid_from) dan satu is_current=true per product_id
-- diperiksa pada staging/load karena partial unique index tidak diasumsikan tersedia.
-- Lookup historis: fact.tanggal_wib >= valid_from AND fact.tanggal_wib < valid_to.
-- Seed awal satu baris per product_id dari snapshot dengan valid_from=1900-01-01.
-- Perubahan snapshot berikutnya menutup versi lama tepat pada tanggal efektif yang teramati;
-- jangan memakai harga_berlaku_dari_sumber untuk menciptakan versi yang tidak ada.
```

```

INSERT INTO dim_product
SELECT -1, 'UNKNOWN', 'Produk tidak diketahui', 'Tidak diketahui', NULL, NULL,
       NULL, DATE '1900-01-01', DATE '9999-12-31', TRUE
WHERE NOT EXISTS (SELECT 1 FROM dim_product WHERE product_sk = -1);

```

20_dim_outlet.sql

Sumber repo: sql/20_dim_outlet.sql

```

-- UTS K6/T2/k9. SCD Type 1: koreksi nama/lokasi outlet menimpa atribut lama.
-- Grain: satu baris per outlet_id. Slice hanya OUT-A, tetapi FK outlet tetap eksplisit.
CREATE SEQUENCE IF NOT EXISTS seq_outlet_sk START 1;
CREATE TABLE IF NOT EXISTS dim_outlet (
    outlet_sk BIGINT PRIMARY KEY,
    outlet_id VARCHAR NOT NULL UNIQUE,
    nama_outlet VARCHAR NOT NULL,
    kota VARCHAR,
    tipe VARCHAR,
    aktif BOOLEAN,
    dibuka_sejak DATE
);
-- Loader memakai nextval('seq_outlet_sk') untuk ID baru dan mempertahankan SK saat Type 1.
INSERT INTO dim_outlet
SELECT -1, 'UNKNOWN', 'Outlet tidak diketahui', NULL, NULL, NULL, NULL
WHERE NOT EXISTS (SELECT 1 FROM dim_outlet WHERE outlet_sk = -1);

```

20_dim_customer.sql

Sumber repo: sql/20_dim_customer.sql

```

-- UTS K6/T2/k9. SCD Type 1: koreksi nama/kota pelanggan menimpa atribut lama.
-- Grain: satu baris per customer_id. Telepon tidak diperlukan untuk keputusan UTS.
-- Customer kosong pada transaksi adalah pelanggan anonim, bukan FK rusak.
CREATE SEQUENCE IF NOT EXISTS seq_customer_sk START 1;
CREATE TABLE IF NOT EXISTS dim_customer (
    customer_sk BIGINT PRIMARY KEY,
    customer_id VARCHAR NOT NULL UNIQUE,
    nama_pelanggan VARCHAR,
    kota VARCHAR,
    tanggal_daftar DATE
);
-- -1 = Unknown untuk ID nonkosong yang tak dapat dicocokkan; -2 = ANONIM untuk ID kosong.
-- Loader memakai nextval('seq_customer_sk') untuk ID baru; koreksi Type 1 tidak mengubah SK.
INSERT INTO dim_customer
SELECT -1, 'UNKNOWN', 'Pelanggan tidak diketahui', NULL, NULL
WHERE NOT EXISTS (SELECT 1 FROM dim_customer WHERE customer_sk = -1);
INSERT INTO dim_customer
SELECT -2, 'ANONIM', 'Pembeli anonim', NULL, NULL
WHERE NOT EXISTS (SELECT 1 FROM dim_customer WHERE customer_sk = -2);

```

30_fact_sales_item.sql

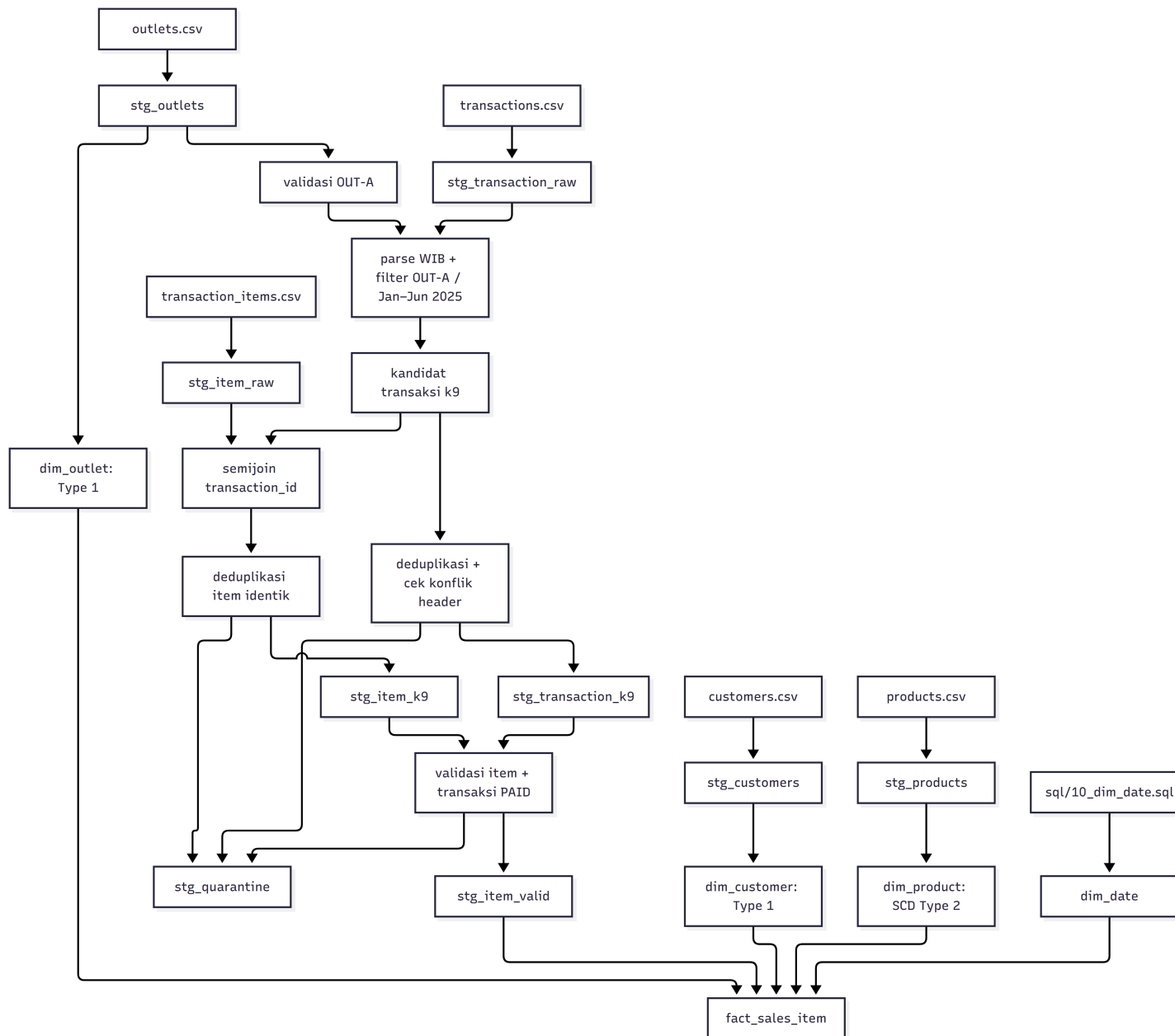
Sumber repo: sql/30_fact_sales_item.sql

```
-- UTS K6/T2/k9. GRAIN: satu baris untuk satu item_id pada satu transaksi OUT-A
-- bertanggal lokal WIB 2025-01-01 s.d. 2025-06-30, setelah deduplikasi identik.
-- Header konflik, item konflik/FK produk hilang, dan transaksi PAID dengan qty negatif
-- masuk karantina staging; tidak masuk fact sampai diperbaiki. REFUND/VOID tetap disimpan
-- untuk hitungan status, tetapi tidak berkontribusi ke nilai_paid_rp.
-- Degenerate dimension: transaction_id; item_id adalah natural key fact/upsert.
CREATE TABLE IF NOT EXISTS fact_sales_item (
  item_id VARCHAR PRIMARY KEY,
  transaction_id VARCHAR NOT NULL,
  outlet_sk BIGINT NOT NULL REFERENCES dim_outlet(outlet_sk),
  date_sk INTEGER NOT NULL,
  product_sk BIGINT NOT NULL REFERENCES dim_product(product_sk),
  customer_sk BIGINT NOT NULL REFERENCES dim_customer(customer_sk),
  waktu_wib TIMESTAMP NOT NULL,
  status_transaksi VARCHAR NOT NULL,
  -- qty_item: NON-ADDITIVE tanpa filter status; refund berisi tanda qty campuran.
  qty_item INTEGER NOT NULL,
  -- harga_satuan_rp: NON-ADDITIVE; harga per unit, jangan SUM.
  harga_satuan_rp DECIMAL(18,2) NOT NULL,
  -- diskon_item_rp: NON-ADDITIVE lintas status; nominal sumber per baris item.
  diskon_item_rp DECIMAL(18,2) NOT NULL,
  -- qty_paid: ADDITIVE pada baris PAID valid; nol untuk REFUND/VOID.
  qty_paid INTEGER NOT NULL,
  -- nilai_paid_rp: ADDITIVE pada tanggal/produk/outlet/pelanggan untuk PAID valid;
  -- 0 untuk REFUND/VOID. Bukan nilai bersih setelah refund atau laba.
  nilai_paid_rp DECIMAL(18,2) NOT NULL,
  CHECK (status_transaksi IN ('PAID','VOID','REFUND')),
  CHECK (harga_satuan_rp > 0),
  CHECK (diskon_item_rp >= 0)
);
-- Kontrak transformasi staging -> fact (desain, bukan perintah load):
--   status=PAID: qty_item > 0, qty_paid=qty_item,
--               nilai_paid_rp=qty_item*harga_satuan_rp-diskon_item_rp.
--   status=REFUND/VOID: qty_paid=0, nilai_paid_rp=0.
--   tanggal_wib=CAST(waktu_wib AS DATE), date_sk=YYYYMMDD dari dim_date.
--   FK date_sk bersifat logis karena generator dim_date yang diberikan memakai CTAS tanpa PK;
--   kontrol load wajib memastikan tepat satu baris dim_date per date_sk.
--   JOIN dim_date, dim_product (satu versi dalam interval), dim_outlet, dim_customer
--   semuanya dari fact; outlet_sk dipilih dari OUT-A pada header tervalidasi.
--   Setiap item_id harus menghasilkan tepat satu baris. Nilai header total_bayar
--   tidak dimasukkan ke fact karena dapat terulang pada setiap item.
```

2. Desain load

Dokumen ini adalah kontrak untuk implementasi sesudah UTS. Tidak ada loader atau warehouse yang dijalankan pada UTS. Sumber hanya lima CSV pada data/raw/t2_umkm/; batas fact adalah OUT-A dan tanggal transaksi lokal WIB [2025-01-01, 2025-07-01).

Alur sumber → staging → dimensi → fact



Alur sumber, staging, dimensi, dan fact; skrip Mermaid tetap tersedia di pipeline/DESIGN_load.md.

Pemetaan waktu. transactions.csv memakai tanggal_waktu, bukan waktu. Jika string berakhir Z, parse sebagai UTC lalu konversi ke Asia/Jakarta; selain itu parse sebagai waktu lokal WIB. Simpan string mentah dan waktu_wib. Nilai gagal parse masuk karantina waktu/cakupan, tidak otomatis masuk atau keluar slice. Filter tanggal dilakukan sesudah konversi. Outlet A berada di Bandung menurut outlets.csv; asumsi WIB untuk string tanpa zona dicatat karena CSV tidak menyediakan kolom zona eksplisit.

Pemetaan item. Item tidak memiliki outlet/tanggal sendiri. Pilih melalui EXISTS ke ID header yang masuk k9, sehingga header duplikat tidak menggandakan item. Item tanpa header tidak dapat dipastikan masuk slice dan dicatat sebagai cakupan tidak diketahui. Jika ID header yang sama ditemukan dengan dua nilai outlet/tanggal/status/total, karantina seluruh ID beserta itemnya sampai sumber dikoreksi; jangan pilih baris secara acak.

Strategi tepat satu per tabel

Tabel	Sumber dan natural key	Strategi terpilih	Partisi/window	Kapan baris dapat tergandakan
stg_outlets	outlets.csv, outlet_id	Full replace baca CSV, validasi satu baris master untuk OUT-A; sumber dim_outlet	Tidak berlaku; master kecil	Duplikat OUT-A atau ID tanpa master diblokir agar lookup outlet_sk fact tidak ambigu.
stg_products	products.csv, product_id	Full replace baca CSV, tipe VARCHAR awal	Tidak berlaku; snapshot master	Produk kembar membuat lookup fact berlipat bila key tidak divalidasi.
stg_customers	customers.csv, customer_id	Full replace baca CSV, tipe VARCHAR awal	Tidak berlaku	Pelanggan kembar dapat menggandakan setiap item terkait.
stg_transaction_raw	transactions.csv, transaction_id	Full replace baca CSV dan parse waktu	Tidak berlaku pada pembacaan	Memuat 34 kelompok ID duplikat pada slice; JOIN polos menggandakan item.
stg_transaction_k9	raw header, transaction_id	Full replace per run: filter, dedup identik, karantina konflik	Outlet A; tanggal WIB [2025-01-01, 2025-07-01)	Jika konflik 4 ID dipilih sembarang, hasil tidak stabil; constraint satu header per ID wajib.
stg_item_raw	transaction_items.csv, item_id	Full replace baca CSV	Tidak berlaku	Sumber memuat 42 kelompok item_id duplikat pada slice.
stg_item_k9	item dengan header k9, item_id	Full replace per run, semijoin header, dedup identik	Himpunan ID transaksi slice, bukan tanggal item	JOIN header mentah atau append item mentah menggandakan baris.
stg_item_valid	item/header kanonik, item_id	Full replace setelah validasi dan karantina	K9 sama seperti header	Jika satu item cocok ke lebih dari satu versi produk, hasil fact berlipat.
stg_quarantine	pelanggaran header/item	Full replace dengan reason code dan key sumber	Satu snapshot input	Append tanpa key (source_file, source_key, reason) menggandakan catatan masalah.
dim_date	generator sql/10_dim_date.sql, date_sk	Full replace sesuai SQL dosen	2024–2027 + Unknown -1	Generator deterministik tidak mengganda; cek key unik setelah dibuat.

Tabel	Sumber dan natural key	Strategi terpilih	Partisi/window	Kapan baris dapat tergandakan
dim_outlet	master outlet, outlet_id	MERGE Type 1: update atribut, insert ID baru	Tidak berlaku	INSERT tiap run membuat outlet ganda; lookup fact wajib tepat satu SK untuk OUT-A.
dim_customer	master pelanggan, customer_id	MERGE Type 1: update atribut, insert ID baru	Tidak berlaku	INSERT tiap run membuat pelanggan ganda; customer kosong memakai anggota ANONIM -2.
dim_product	master produk, product_id + valid_from untuk versi	MERGE SCD Type 2 sesuai perubahan atribut terlacak	Window versi [valid_from, valid_to); bukan watermark transaksi	Menambah versi setiap run tanpa perubahan atau interval tumpang tindih menggandakan lookup.
fact_sales_item	item valid, item_id; outlet_id header kanonik untuk lookup outlet_sk	MERGE by item_id, lalu hapus target k9 yang tak ada pada sumber valid dalam transaksi atomik	outlet_id=OUT-A + tanggal WIB [2025-01-01, 2025-07-01)	INSERT tiap run, JOIN header mentah, atau lookup dua versi produk menggandakan nilai; FK outlet dan filter k9 menjaga cakupan outlet.

Kontrak validasi dan deduplikasi

- Semua sumber dibaca sebagai teks sebelum cast. Trim status ke uppercase; domain teramati PAID, VOID, REFUND. Nilai lain dikarantina sampai pemilik data menetapkan makna. item_id dan transaction_id kosong, gagal cast qty/harga/diskon, harga ≤ 0 , diskon < 0 , dan tanggal tak terparse adalah blocking untuk baris terkait.
- Satu header kanonik per transaction_id. Duplikat identik dibuang deterministik; empat ID konflik pada k9 masuk stg_quarantine beserta semua itemnya. Satu item kanonik per item_id; 42 kelompok duplikat item yang identik boleh dikurangi menjadi satu. Jika item_id sama dengan payload berbeda, karantina seluruh ID.
- Qty negatif pada header PAID adalah blocking. Karantina **seluruh transaksi PAID** yang terkena agar angka item yang tersisa tidak memberi total parsial yang menyesatkan. Profil sumber menunjukkan 92 baris item negatif PAID pada 90 transaksi, sedangkan 2 baris negatif VOID dicatat sebagai warning karena VOID bernilai_paid_rp=0. VOID dan REFUND tetap disimpan untuk hitungan status, dengan nilai_paid_rp = 0.
- Item dengan product_id nonkosong yang tidak ada di master adalah blocking/karantina, tidak disamarkan menjadi produk Unknown untuk nilai penjualan. customer_id kosong berarti ANONIM customer_sk=-2; customer ID nonkosong yang tidak cocok memakai Unknown -1 disertai warning dan daftar audit. Outlet selain A tidak masuk slice; OUT-A harus tepat satu pada master; outlet_sk fact di-lookup dari dim_outlet memakai outlet_id header kanonik.
- Asumsi kerja diskon adalah **nominal rupiah per baris item**, didukung nilai 0/1.000/2.000/5.000 dan tidak ada nilai yang melebihi subtotal positif pada slice. Sumber tidak mendokumentasikan satuannya; konfirmasi diperlukan sebelum menyebut angka penjualan sebagai nilai finansial resmi. Untuk baris PAID valid, nilai_paid_rp = qty × harga_satuan - diskon, qty_paid = qty. Tolak subtotal negatif. Untuk REFUND/VOID, nilai_paid_rp=0 dan qty_paid=0; simpan qty/harga/diskon mentah untuk audit status.
- total_bayar header **tidak** disalin ke fact item. Profil menunjukkan selisih terhadap jumlah item pada banyak transaksi; rekonsiliasi sumber-vs-fact adalah warning investigasi, bukan aturan menimpa harga item. Hitung transaksi dari COUNT(DISTINCT transaction_id) pada fact valid.

SCD produk dan lookup

Snapshot products.csv berisi satu baris per product_id. harga_berlaku_dari pada sebagian besar produk berada setelah awal 2025, sehingga bukan bukti semua versi harga historis. Untuk seed awal, simpan **atribut yang diketahui dari snapshot** sebagai satu versi per produk dengan valid_from=1900-01-01, valid_to=9999-12-31, is_current=true. Tanggal sentinel hanya memungkinkan lookup historis yang konsisten; nama/kategori masa lalu tetap berlabel *as known from current snapshot*. Nilai penjualan

memakai harga **item**, sehingga tidak mengasumsikan harga referensi master saat transaksi.

Ketika snapshot produk berikutnya tersedia, bandingkan hash atribut terlacak (nama_produk, kategori, harga_satuan referensi, aktif). Jika tidak berubah, jangan buat versi. Jika berubah, tutup versi aktif pada tanggal perubahan yang **teramati**, buat SK baru dengan valid_from sama dengan batas penutupan, dan pastikan tepat satu is_current=true per product_id. Perubahan yang diketahui terlambat memerlukan aturan restatement fact berdasarkan tanggal efektif yang benar-benar diketahui; jangan membuat histori masa lalu dari tebakan. product_sk dialokasikan dari sequence, bukan dihitung ulang dengan row_number() pada setiap load. customer_sk dan outlet_sk stabil saat Type 1 menimpa atribut; outlet tetap punya surrogate key meskipun slice hanya OUT-A.

Lookup fact ke dim_product memakai product_id serta tanggal_wib >= valid_from AND tanggal_wib < valid_to; wajib menghasilkan tepat satu SK. dim_customer dan dim_outlet lookup dari key bisnis; fact menyimpan date_sk, product_sk, outlet_sk, dan customer_sk. Cakupan OUT-A dijaga dari header kanonik dan FK outlet. dim_date.date_sk dari tanggal WIB; cek key kalender unik walaupun generator bawaan memakai CTAS tanpa deklarasi PK.

Upsert, perubahan, dan idempotensi

Natural key fact adalah item_id, setelah profil memastikan 42 duplikatnya identik. MERGE mengubah nilai dan FK bila item/header yang sama dikoreksi; INSERT hanya untuk item_id baru. Setelah MERGE, hapus item target **dalam slice k9** yang tidak lagi ada pada stg_item_valid karena pembatalan, perubahan tanggal/outlet, atau karantina. Operasi dilakukan atomik agar pembaca tidak melihat snapshot setengah jadi. Jangan menghapus record di luar k9.

Partisi logis adalah bulan transaksi WIB, tetapi **tidak ada updated_at** pada sumber yang menjamin koreksi lama tertangkap oleh watermark. Oleh karena itu setiap run membaca ulang seluruh enam bulan k9 sebelum MERGE; window tidak dipersempit hanya ke bulan terbaru. Setelah input yang sama diproses dua kali, jumlah item_id, total nilai_paid_rp, dan jumlah versi produk harus tetap sama. Cek ini adalah kriteria desain untuk implementasi nanti, bukan klaim hasil UTS.

Urutan dependency dan rekonsiliasi

1. Baca snapshot lima CSV ke staging dan catat jumlah/hash sumber.
2. Normalisasi waktu/status/tipe; tetapkan transaksi k9; pisahkan duplikat identik, konflik, dan baris tak terparse.
3. Bentuk item k9 lewat semijoin, dedup, validasi produk/qty/diskon, dan karantina transaksi PAID bermasalah.
4. Siapkan dim_date, lalu MERGE dim_outlet (Type 1), dim_customer (Type 1), dan dim_product (Type 2); validasi OUT-A pada master sebelum lookup fact.
5. Lookup tepat satu SK per dimensi dan MERGE/hapus terbatas pada fact_sales_item k9.
6. Bandingkan jumlah item valid terhadap fact, hitung ID unik/duplikat, pastikan tidak ada SK NULL atau versi produk ganda, lalu catat selisih total_bayar header sebagai warning untuk diteliti.

Dimensi harus siap sebelum fact agar setiap FK dapat diisi. Baris Unknown/ANONIM harus dibuat sekali saja dan tidak berubah saat rerun. Semua kontrol ini dirancang untuk implementasi setelah UTS; tidak ada load yang dieksekusi sekarang.

3. Enam test kualitas dan bukti profiling

Definisi test T2/k9

Sumber repo: tests/test_definitions.yml, hanya enam entri topic: t2. expected menyatakan hasil pada sumber, bukan eksekusi test runner.

tests:

```
- name: transaction_id_duplikat_k9
  topic: t2
  severity: blocking
  scope: source
  sql: >-
    WITH slice AS (SELECT t.* FROM raw_transactions t WHERE t.outlet_id = 'OUT-A' AND CAST(CASE WHEN
    upper(trim(t.tanggal_waktu)) LIKE '%Z' THEN TRY_CAST(t.tanggal_waktu AS TIMESTAMPTZ) AT TIME ZONE
    'Asia/Jakarta' ELSE TRY_CAST(t.tanggal_waktu AS TIMESTAMP) END AS DATE) >= DATE '2025-01-01' AND
    CAST(CASE WHEN upper(trim(t.tanggal_waktu)) LIKE '%Z' THEN TRY_CAST(t.tanggal_waktu AS TIMESTAMPTZ) AT
    TIME ZONE 'Asia/Jakarta' ELSE TRY_CAST(t.tanggal_waktu AS TIMESTAMP) END AS DATE) < DATE '2025-07-01')
    SELECT count(*) FROM (SELECT transaction_id FROM slice GROUP BY 1 HAVING count(*) > 1)
  expected: fail
  perkiraan_baris: 34
  justifikasi: ID transaksi kembar pada Outlet A/periode menyebabkan item ter-join berulang; unitnya kelompok ID.
- name: transaction_id_konflik_k9
  topic: t2
  severity: blocking
  scope: source
  sql: >-
    WITH slice AS (SELECT t.* FROM raw_transactions t WHERE t.outlet_id = 'OUT-A' AND CAST(CASE WHEN
    upper(trim(t.tanggal_waktu)) LIKE '%Z' THEN TRY_CAST(t.tanggal_waktu AS TIMESTAMPTZ) AT TIME ZONE
    'Asia/Jakarta' ELSE TRY_CAST(t.tanggal_waktu AS TIMESTAMP) END AS DATE) >= DATE '2025-01-01' AND
    CAST(CASE WHEN upper(trim(t.tanggal_waktu)) LIKE '%Z' THEN TRY_CAST(t.tanggal_waktu AS TIMESTAMPTZ) AT
    TIME ZONE 'Asia/Jakarta' ELSE TRY_CAST(t.tanggal_waktu AS TIMESTAMP) END AS DATE) < DATE '2025-07-01')
    SELECT count(*) FROM (SELECT transaction_id FROM slice GROUP BY 1 HAVING count(DISTINCT
    concat_ws('|',coalesce(outlet_id,''),coalesce(customer_id,''),coalesce(tanggal_waktu,''),coalesce(status,''),coalesce(total_bayar,'')))
    > 1)
  expected: fail
  perkiraan_baris: 4
  justifikasi: Header dengan ID sama tetapi atribut berbeda tidak punya aturan pilih yang aman dan harus dikarantina; unitnya ID.
- name: item_id_duplikat_k9
  topic: t2
  severity: blocking
  scope: source
  sql: >-
    WITH slice AS (SELECT t.* FROM raw_transactions t WHERE t.outlet_id = 'OUT-A' AND CAST(CASE WHEN
    upper(trim(t.tanggal_waktu)) LIKE '%Z' THEN TRY_CAST(t.tanggal_waktu AS TIMESTAMPTZ) AT TIME ZONE
    'Asia/Jakarta' ELSE TRY_CAST(t.tanggal_waktu AS TIMESTAMP) END AS DATE) >= DATE '2025-01-01' AND
    CAST(CASE WHEN upper(trim(t.tanggal_waktu)) LIKE '%Z' THEN TRY_CAST(t.tanggal_waktu AS TIMESTAMPTZ) AT
    TIME ZONE 'Asia/Jakarta' ELSE TRY_CAST(t.tanggal_waktu AS TIMESTAMP) END AS DATE) < DATE '2025-07-01'),
    item_slice AS (SELECT i.* FROM raw_transaction_items i WHERE EXISTS (SELECT 1 FROM slice t WHERE
    t.transaction_id=i.transaction_id)) SELECT count(*) FROM (SELECT item_id FROM item_slice GROUP BY 1
    HAVING count(*) > 1)
  expected: fail
  perkiraan_baris: 42
```

```

justifikasi: Item ID kembar menggandakan nilai penjualan; unitnya kelompok item_id sebelum deduplikasi.
- name: qty_negatif_paid_k9
  topic: t2
  severity: blocking
  scope: source
  sql: >-
    WITH slice AS (SELECT t.* FROM raw_transactions t WHERE t.outlet_id = 'OUT-A' AND CAST(CASE WHEN
    upper(trim(t.tanggal_waktu)) LIKE '%Z' THEN TRY_CAST(t.tanggal_waktu AS TIMESTAMPTZ) AT TIME ZONE
    'Asia/Jakarta' ELSE TRY_CAST(t.tanggal_waktu AS TIMESTAMP) END AS DATE) >= DATE '2025-01-01' AND
    CAST(CASE WHEN upper(trim(t.tanggal_waktu)) LIKE '%Z' THEN TRY_CAST(t.tanggal_waktu AS TIMESTAMPTZ) AT
    TIME ZONE 'Asia/Jakarta' ELSE TRY_CAST(t.tanggal_waktu AS TIMESTAMP) END AS DATE) < DATE '2025-07-01')
    SELECT count(*) FROM raw_transaction_items i WHERE TRY_CAST(i.qty AS INTEGER) < 0 AND EXISTS (SELECT 1
    FROM slice t WHERE t.transaction_id=i.transaction_id AND upper(trim(t.status)) = 'PAID')
  expected: fail
  perkiraan_baris: 92
  justifikasi: Qty negatif pada transaksi PAID membuat nilai item tidak layak dijumlah; unitnya baris item mentah, tanpa JOIN pengganda.
- name: transaksi_tanpa_item_k9
  topic: t2
  severity: warning
  scope: source
  sql: >-
    WITH slice AS (SELECT t.* FROM raw_transactions t WHERE t.outlet_id = 'OUT-A' AND CAST(CASE WHEN
    upper(trim(t.tanggal_waktu)) LIKE '%Z' THEN TRY_CAST(t.tanggal_waktu AS TIMESTAMPTZ) AT TIME ZONE
    'Asia/Jakarta' ELSE TRY_CAST(t.tanggal_waktu AS TIMESTAMP) END AS DATE) >= DATE '2025-01-01' AND
    CAST(CASE WHEN upper(trim(t.tanggal_waktu)) LIKE '%Z' THEN TRY_CAST(t.tanggal_waktu AS TIMESTAMPTZ) AT
    TIME ZONE 'Asia/Jakarta' ELSE TRY_CAST(t.tanggal_waktu AS TIMESTAMP) END AS DATE) < DATE '2025-07-01')
    SELECT count(*) FROM slice t WHERE NOT EXISTS (SELECT 1 FROM raw_transaction_items i WHERE
    i.transaction_id=t.transaction_id)
  expected: fail
  perkiraan_baris: 21
  justifikasi: Header tanpa item membuat total header tidak bisa dijelaskan pada grain fact item; unitnya baris header.
- name: item_produk_orphan_k9
  topic: t2
  severity: blocking
  scope: source
  sql: >-
    WITH slice AS (SELECT t.* FROM raw_transactions t WHERE t.outlet_id = 'OUT-A' AND CAST(CASE WHEN
    upper(trim(t.tanggal_waktu)) LIKE '%Z' THEN TRY_CAST(t.tanggal_waktu AS TIMESTAMPTZ) AT TIME ZONE
    'Asia/Jakarta' ELSE TRY_CAST(t.tanggal_waktu AS TIMESTAMP) END AS DATE) >= DATE '2025-01-01' AND
    CAST(CASE WHEN upper(trim(t.tanggal_waktu)) LIKE '%Z' THEN TRY_CAST(t.tanggal_waktu AS TIMESTAMPTZ) AT
    TIME ZONE 'Asia/Jakarta' ELSE TRY_CAST(t.tanggal_waktu AS TIMESTAMP) END AS DATE) < DATE '2025-07-01')
    SELECT count(*) FROM raw_transaction_items i WHERE EXISTS (SELECT 1 FROM slice t WHERE
    t.transaction_id=i.transaction_id) AND NOT EXISTS (SELECT 1 FROM raw_products p WHERE
    p.product_id=i.product_id)
  expected: pass
  perkiraan_baris: 0
  justifikasi: Item tanpa produk tidak bisa dipetakan ke dimensi produk; unitnya baris item mentah.

```

Bukti angka dari CSV slice k9

Sumber hanya lima CSV pada data/raw/t2_umkm/. Semua query di bawah merupakan SELECT pada view baca-saja; tidak ada warehouse yang dibangun. Waktu bertanda Z diparse sebagai UTC dan dikonversi ke WIB (Asia/Jakarta); waktu tanpa zona diasumsikan WIB. Filter memakai tanggal WIB [2025-01-01, 2025-07-01) dan outlet_id = OUT-A.

Ringkasan cakupan

Pemeriksaan	Hasil	Catatan
Header seluruh sumber	15.892	Profil bawaan; mencakup outlet lain
Header slice	2.578	2.543 ID transaksi unik
Item terkait slice	5.074	5.032 ID item unik; dipilih dengan EXISTS, bukan JOIN
Header bertanda Z pada slice	375	0 timestamp gagal parse pada seluruh sumber
Rentang transaksi slice	1 Jan–30 Jun 2025	Berdasarkan WIB
Header PAID/VOID/REFUND	2.504 / 43 / 31	PAID mencakup kapitalisasi berbeda

Enam query yang menjadi bukti perkiraan_baris

Definisi raw_transactions, raw_transaction_items, raw_products sama dengan view baca-saja dari CSV (read_csv_auto(..., all_varchar=true)). Query di sini identik dengan SQL test YAML; hasil adalah jumlah pelanggaran awal sumber dan bukan hasil sesudah penanganan.

transaction_id_duplikat_k9

- Hasil: **34**; expected: fail, severity blocking.
- Unit dan alasan: ID transaksi kembar pada Outlet A/periode menyebabkan item ter-join berulang; unitnya kelompok ID.

```
WITH slice AS (SELECT t.* FROM raw_transactions t WHERE t.outlet_id = 'OUT-A' AND CAST(CASE WHEN upper(trim(t.tanggal_waktu)) LIKE '%Z' THEN TRY_CAST(t.tanggal_waktu AS TIMESTAMPTZ) AT TIME ZONE 'Asia/Jakarta' ELSE TRY_CAST(t.tanggal_waktu AS TIMESTAMP) END AS DATE) >= DATE '2025-01-01' AND CAST(CASE WHEN upper(trim(t.tanggal_waktu)) LIKE '%Z' THEN TRY_CAST(t.tanggal_waktu AS TIMESTAMPTZ) AT TIME ZONE 'Asia/Jakarta' ELSE TRY_CAST(t.tanggal_waktu AS TIMESTAMP) END AS DATE) < DATE '2025-07-01') SELECT count(*) FROM (SELECT transaction_id FROM slice GROUP BY 1 HAVING count(*) > 1);
```

transaction_id_konflik_k9

- Hasil: **4**; expected: fail, severity blocking.
- Unit dan alasan: Header dengan ID sama tetapi atribut berbeda tidak punya aturan pilih yang aman dan harus dikarantina; unitnya ID.

```
WITH slice AS (SELECT t.* FROM raw_transactions t WHERE t.outlet_id = 'OUT-A' AND CAST(CASE WHEN upper(trim(t.tanggal_waktu)) LIKE '%Z' THEN TRY_CAST(t.tanggal_waktu AS TIMESTAMPTZ) AT TIME ZONE 'Asia/Jakarta' ELSE TRY_CAST(t.tanggal_waktu AS TIMESTAMP) END AS DATE) >= DATE '2025-01-01' AND CAST(CASE WHEN upper(trim(t.tanggal_waktu)) LIKE '%Z' THEN TRY_CAST(t.tanggal_waktu AS TIMESTAMPTZ) AT TIME ZONE 'Asia/Jakarta' ELSE TRY_CAST(t.tanggal_waktu AS TIMESTAMP) END AS DATE) < DATE '2025-07-01') SELECT count(*) FROM (SELECT transaction_id FROM slice GROUP BY 1 HAVING count(DISTINCT concat_ws(' ', coalesce(outlet_id, ''), coalesce(customer_id, ''), coalesce(tanggal_waktu, ''), coalesce(status, ''), coalesce(total_bayar, ''))) > 1);
```

item_id_duplikat_k9

- Hasil: **42**; expected: fail, severity blocking.
- Unit dan alasan: Item ID kembar menggandakan nilai penjualan; unitnya kelompok item_id sebelum deduplikasi.

```
WITH slice AS (SELECT t.* FROM raw_transactions t WHERE t.outlet_id = 'OUT-A' AND CAST(CASE WHEN upper(trim(t.tanggal_waktu)) LIKE '%Z' THEN TRY_CAST(t.tanggal_waktu AS TIMESTAMPTZ) AT TIME ZONE 'Asia/Jakarta' ELSE TRY_CAST(t.tanggal_waktu AS TIMESTAMP) END AS DATE) >= DATE '2025-01-01' AND CAST(CASE WHEN upper(trim(t.tanggal_waktu)) LIKE '%Z' THEN TRY_CAST(t.tanggal_waktu AS TIMESTAMPTZ) AT TIME ZONE 'Asia/Jakarta' ELSE TRY_CAST(t.tanggal_waktu AS TIMESTAMP) END AS DATE) < DATE '2025-07-01'), item_slice AS (SELECT i.* FROM raw_transaction_items i WHERE EXISTS (SELECT 1 FROM slice t WHERE t.transaction_id=i.transaction_id))
SELECT count(*) FROM (SELECT item_id FROM item_slice GROUP BY 1 HAVING count(*) > 1);
```

qty_negatif_paid_k9

- Hasil: **92**; expected: fail, severity blocking.
- Unit dan alasan: Qty negatif pada transaksi PAID membuat nilai item tidak layak dijumlah; unitnya baris item mentah, tanpa JOIN pengganda.

```
WITH slice AS (SELECT t.* FROM raw_transactions t WHERE t.outlet_id = 'OUT-A' AND CAST(CASE WHEN upper(trim(t.tanggal_waktu)) LIKE '%Z' THEN TRY_CAST(t.tanggal_waktu AS TIMESTAMPTZ) AT TIME ZONE 'Asia/Jakarta' ELSE TRY_CAST(t.tanggal_waktu AS TIMESTAMP) END AS DATE) >= DATE '2025-01-01' AND CAST(CASE WHEN upper(trim(t.tanggal_waktu)) LIKE '%Z' THEN TRY_CAST(t.tanggal_waktu AS TIMESTAMPTZ) AT TIME ZONE 'Asia/Jakarta' ELSE TRY_CAST(t.tanggal_waktu AS TIMESTAMP) END AS DATE) < DATE '2025-07-01') SELECT count(*) FROM raw_transaction_items i WHERE TRY_CAST(i.qty AS INTEGER) < 0 AND EXISTS (SELECT 1 FROM slice t WHERE t.transaction_id=i.transaction_id AND upper(trim(t.status)) = 'PAID');
```

transaksi_tanpa_item_k9

- Hasil: **21**; expected: fail, severity warning.
- Unit dan alasan: Header tanpa item membuat total header tidak bisa dijelaskan pada grain fact item; unitnya baris header.

```
WITH slice AS (SELECT t.* FROM raw_transactions t WHERE t.outlet_id = 'OUT-A' AND CAST(CASE WHEN upper(trim(t.tanggal_waktu)) LIKE '%Z' THEN TRY_CAST(t.tanggal_waktu AS TIMESTAMPTZ) AT TIME ZONE 'Asia/Jakarta' ELSE TRY_CAST(t.tanggal_waktu AS TIMESTAMP) END AS DATE) >= DATE '2025-01-01' AND CAST(CASE WHEN upper(trim(t.tanggal_waktu)) LIKE '%Z' THEN TRY_CAST(t.tanggal_waktu AS TIMESTAMPTZ) AT TIME ZONE 'Asia/Jakarta' ELSE TRY_CAST(t.tanggal_waktu AS TIMESTAMP) END AS DATE) < DATE '2025-07-01') SELECT count(*) FROM slice t WHERE NOT EXISTS (SELECT 1 FROM raw_transaction_items i WHERE i.transaction_id=t.transaction_id);
```

item_produk_orphan_k9

- Hasil: **0**; expected: pass, severity blocking.
- Unit dan alasan: Item tanpa produk tidak bisa dipetakan ke dimensi produk; unitnya baris item mentah.

```
WITH slice AS (SELECT t.* FROM raw_transactions t WHERE t.outlet_id = 'OUT-A' AND CAST(CASE WHEN upper(trim(t.tanggal_waktu)) LIKE '%Z' THEN TRY_CAST(t.tanggal_waktu AS TIMESTAMPTZ) AT TIME ZONE 'Asia/Jakarta' ELSE TRY_CAST(t.tanggal_waktu AS TIMESTAMP) END AS DATE) >= DATE '2025-01-01' AND CAST(CASE WHEN upper(trim(t.tanggal_waktu)) LIKE '%Z' THEN TRY_CAST(t.tanggal_waktu AS TIMESTAMPTZ) AT TIME ZONE 'Asia/Jakarta' ELSE TRY_CAST(t.tanggal_waktu AS TIMESTAMP) END AS DATE) < DATE '2025-07-01') SELECT count(*) FROM raw_transaction_items i WHERE EXISTS (SELECT 1 FROM slice t WHERE t.transaction_id=i.transaction_id) AND NOT EXISTS (SELECT 1 FROM raw_products p WHERE p.product_id=i.product_id);
```

Temuan tambahan untuk desain

- Duplikasi header slice: 34 ID, 35 baris berlebih. Empat ID memiliki atribut header yang saling bertentangan; jangan memilih salah satunya secara arbitrer.
- Duplikasi item: 42 ID, 42 baris berlebih; seluruh 42 kelompok tersebut identik pada kolom CSV, sehingga boleh dide-duplikasi deterministik.

- Ada 92 baris item qty negatif pada transaksi PAID dan 2 pada VOID. Sebanyak 90 transaksi PAID berkaitan dengan 212 item unik; seluruh transaksi PAID tersebut dikarantina agar nilainya tidak parsial. VOID tetap disimpan dengan nilai_paid_rp=0 dan anomali tanda dicatat sebagai warning.
- Ada 21 header slice tanpa item. Orphan produk pada item slice = 0, harga item <= 0 = 0, gagal cast harga/qty/diskon = 0.
- Diskon item hanya bernilai 0, 1.000, 2.000, dan 5.000; tidak ada nilai yang melebihi subtotal positif. Ini mendukung asumsi diskon nominal per item, tetapi metadata sumber tidak menyatakannya secara eksplisit.
- Harga master produk mempunyai satu baris per product_id, sementara 110/120 baris memiliki harga_berlaku_dari setelah 1 Januari 2025. Snapshot ini tidak membuktikan histori harga awal periode; gunakan harga pada item transaksi untuk nilai penjualan.
- Banyak REFUND memuat qty positif. Status tersebut belum cukup untuk menghitung nilai retur bersih; laporkan jumlah transaksi REFUND secara terpisah dari nilai item PAID.
- total_bayar header sering tidak sama dengan penjumlahan qty × harga_satuan – diskon pada item. Karena itu, nilai header tidak boleh disalin ke setiap item; rekonsiliasi menjadi warning desain, bukan persamaan yang diasumsikan benar.
- Satu item mentah tanpa header di seluruh sumber tidak bisa secara pasti dikaitkan ke slice karena tidak ada outlet/tanggal pada item itu sendiri.

Cara mereproduksi

Jalankan profiling bawaan melalui `.venv/bin/python -m pipeline.profile --topic t2 --slice k9 --json`, lalu jalankan query SQL di atas pada view CSV baca-saja dengan DuckDB di memori. Profil bawaan belum menerapkan filter slice; angka enam test berasal dari query pada dokumen ini.

Validasi analitik dari CSV (baca-saja)

Angka berikut dihitung dengan kebijakan pada `pipeline/DESIGN_load.md`: buang duplikat header/item yang identik, karantina 4 ID header konflik, dan keluarkan seluruh transaksi PAID yang mempunyai qty negatif. Seluruh keluaran fact di bawah adalah **perkiraan desain berbasis CSV**, bukan hasil menjalankan warehouse.

Bulan WIB	Nilai item PAID valid (Rp)	ID PAID valid	ID REFUND pada fact	Semua ID pada fact
2025-01	30.193.000	398	4	408
2025-02	27.861.000	358	5	368
2025-03	27.420.000	353	6	370
2025-04	32.294.000	429	5	439
2025-05	33.051.000	413	1	423
2025-06	30.959.000	405	8	420
Total	181.778.000	2.356	29	2.428

Produk peringkat pertama menurut nilai item PAID: Januari PRD-0093 (Rp841.000); Februari PRD-0003 (Rp715.000); Maret PRD-0056 (Rp882.000); April PRD-0053 (Rp1.155.000); Mei PRD-0003 (Rp841.000); Juni PRD-0003 (Rp888.000). Jumlah item fact yang diperkirakan: 4.801, terdiri dari 4.656 PAID, 55 REFUND, dan 90 VOID. Angka ini perlu direkonsiliasi ketika implementasi diperbolehkan.

Query reproduksi menggunakan CTE tx untuk parse waktu, slice untuk filter k9, canonical_tx untuk menolak ID konflik, canonical_item untuk SELECT DISTINCT item, invalid_paid untuk transaksi PAID ber-qty negatif, dan joined untuk agregasi. Rumus PAID: $qty * harga_satuan - diskon$. SQL query lengkap dapat dibuat ulang dari definisi tahap 1 pada PLAN; angka di atas bukan hasil query fact.

SQL reproduksi angka analitik

Kedua query berikut langsung membaca CSV melalui view raw_transactions dan raw_transaction_items seperti di atas. Mereka hanya digunakan sebagai profiling baca-saja untuk jawaban harapan; belum mengeksekusi SQL fact.

Tren dan rasio status:

```
WITH tx AS (
  SELECT *, CASE WHEN upper(trim(tanggal_waktu)) LIKE '%Z'
    THEN TRY_CAST(tanggal_waktu AS TIMESTAMPTZ) AT TIME ZONE 'Asia/Jakarta'
    ELSE TRY_CAST(tanggal_waktu AS TIMESTAMP) END AS waktu_wib
  FROM raw_transactions
), slice AS (
  SELECT * FROM tx WHERE outlet_id='OUT-A'
    AND CAST(waktu_wib AS DATE) >= DATE '2025-01-01'
    AND CAST(waktu_wib AS DATE) < DATE '2025-07-01'
), canonical_tx AS (
  SELECT transaction_id, min(waktu_wib) AS waktu_wib,
    upper(trim(min(status))) AS status
  FROM slice GROUP BY 1
  HAVING count(DISTINCT concat_ws('|', coalesce(outlet_id, ''),
    coalesce(customer_id, ''), coalesce(tanggal_waktu, ''),
    coalesce(status, ''), coalesce(total_bayar, ''))) = 1
), canonical_item AS (
  SELECT DISTINCT i.* FROM raw_transaction_items i
  WHERE EXISTS (SELECT 1 FROM canonical_tx t
    WHERE t.transaction_id = i.transaction_id)
), invalid_paid AS (
  SELECT DISTINCT t.transaction_id FROM canonical_tx t
  JOIN canonical_item i USING(transaction_id)
  WHERE t.status='PAID' AND TRY_CAST(i.qty AS INT) < 0
), joined AS (
  SELECT t.transaction_id, t.waktu_wib, t.status, i.item_id, i.product_id,
    TRY_CAST(i.qty AS INT) AS qty,
    TRY_CAST(i.harga_satuan AS INT) AS harga,
    TRY_CAST(i.diskon AS INT) AS diskon
  FROM canonical_tx t JOIN canonical_item i USING(transaction_id)
  WHERE NOT EXISTS (SELECT 1 FROM invalid_paid x
    WHERE x.transaction_id = t.transaction_id)
)
SELECT strftime(waktu_wib, '%Y-%m') AS bulan,
  sum(CASE WHEN status='PAID' THEN qty*harga-diskon ELSE 0 END) AS nilai_paid_rp,
  count(DISTINCT CASE WHEN status='PAID' THEN transaction_id END) AS tx_paid,
  count(DISTINCT CASE WHEN status='REFUND' THEN transaction_id END) AS tx_refund,
  count(DISTINCT transaction_id) AS tx_tercatat
FROM joined GROUP BY 1 ORDER BY 1;
```

Produk peringkat pertama per bulan:

```

WITH tx AS (
    SELECT *, CASE WHEN upper(trim(tanggal_waktu)) LIKE '%Z'
    THEN TRY_CAST(tanggal_waktu AS TIMESTAMPTZ) AT TIME ZONE 'Asia/Jakarta'
    ELSE TRY_CAST(tanggal_waktu AS TIMESTAMP) END AS waktu_wib
    FROM raw_transactions
), slice AS (
    SELECT * FROM tx WHERE outlet_id='OUT-A'
    AND CAST(waktu_wib AS DATE)>=DATE '2025-01-01'
    AND CAST(waktu_wib AS DATE)<DATE '2025-07-01'
), canonical_tx AS (
    SELECT transaction_id, min(waktu_wib) AS waktu_wib,
    upper(trim(min(status))) AS status
    FROM slice GROUP BY 1
    HAVING count(DISTINCT concat_ws('|',coalesce(outlet_id,''),
    coalesce(customer_id,''),coalesce(tanggal_waktu,''),
    coalesce(status,''),coalesce(total_bayar,''))=1
), canonical_item AS (
    SELECT DISTINCT i.* FROM raw_transaction_items i
    WHERE EXISTS (SELECT 1 FROM canonical_tx t
    WHERE t.transaction_id=i.transaction_id)
), invalid_paid AS (
    SELECT DISTINCT t.transaction_id FROM canonical_tx t
    JOIN canonical_item i USING(transaction_id)
    WHERE t.status='PAID' AND TRY_CAST(i.qty AS INT)<0
), joined AS (
    SELECT t.transaction_id,t.waktu_wib,t.status,i.item_id,i.product_id,
    TRY_CAST(i.qty AS INT) AS qty,
    TRY_CAST(i.harga_satuan AS INT) AS harga,
    TRY_CAST(i.diskon AS INT) AS diskon
    FROM canonical_tx t JOIN canonical_item i USING(transaction_id)
    WHERE NOT EXISTS (SELECT 1 FROM invalid_paid x
    WHERE x.transaction_id=t.transaction_id)
), penjualan_produk AS (
    SELECT strftime(waktu_wib,'%Y-%m') AS bulan,product_id,
    sum(qty*harga-diskon) AS nilai_paid_rp
    FROM joined WHERE status='PAID' GROUP BY 1,2
), peringkat AS (
    SELECT *,dense_rank() OVER(PARTITION BY bulan ORDER BY nilai_paid_rp DESC) AS urutan
    FROM penjualan_produk
)
SELECT bulan,product_id,nilai_paid_rp FROM peringkat WHERE urutan=1 ORDER BY bulan;

```

4. Tiga query analitik dan jawaban harapan

Jawaban harapan satu kalimat untuk masing-masing query tercatat pada brief di bagian 1; angkanya berasal dari profiling CSV baca-saja, bukan warehouse.

q01_tren_bulanan.sql

Sumber repo: [sql/40_analytics/q01_tren_bulanan.sql](#)

```
-- UTS K6/T2/k9 – Q01: Berapa nilai item PAID valid dan transaksi PAID per bulan?
-- Keluaran: bulan | nilai_paid_rp | transaksi_paid.
-- Grain keluaran: satu bulan kalender WIB untuk OUT-A, Januari–Juni 2025.
-- Jawaban harapan: Mei 2025 tertinggi dengan Rp33.051.000 dari 413 transaksi PAID valid.
-- Bukti: docs/profile_t2_k9_slice.md, bagian validasi analitik; query fact ini belum dijalankan.
-- Bulan tanpa baris fact menghasilkan NULL; bulan dengan fact hanya REFUND/VOID menghasilkan 0.
WITH bulan AS (
    SELECT strftime(d, '%Y-%m') AS bulan
    FROM unnest(generate_series(DATE '2025-01-01', DATE '2025-06-01', INTERVAL 1 MONTH)) AS x(d)
), agregat AS (
    SELECT strftime(d.full_date, '%Y-%m') AS bulan,
           sum(f.nilai_paid_rp) AS nilai_paid_rp,
           count(DISTINCT CASE WHEN f.status_transaksi = 'PAID' THEN f.transaction_id END) AS transaksi_paid
    FROM fact_sales_item f
    JOIN dim_date d ON d.date_sk = f.date_sk
    JOIN dim_outlet o ON o.outlet_sk = f.outlet_sk
    WHERE o.outlet_id = 'OUT-A'
           AND d.full_date >= DATE '2025-01-01' AND d.full_date < DATE '2025-07-01'
    GROUP BY 1
)
SELECT b.bulan, a.nilai_paid_rp, a.transaksi_paid
FROM bulan b LEFT JOIN agregat a USING (bulan)
ORDER BY b.bulan;
```

q02.sql

Sumber repo: [sql/40_analytics/q02.sql](#)

```
-- UTS K6/T2/k9 – Q02: Produk apa yang paling besar kontribusi nilai PAID tiap bulan?
-- Keluaran: bulan | product_id | nama_produk_tercatat | nilai_paid_rp | peringkat.
-- Grain keluaran: satu identitas produk per bulan WIB; versi SCD digabung menurut product_id.
-- Jawaban harapan: PRD-0003 memimpin pada Februari, Mei, dan Juni 2025 setelah aturan kualitas diterapkan.
-- Bukti: docs/profile_t2_k9_slice.md, bagian validasi analitik; query fact ini belum dijalankan.
WITH penjualan_produk AS (
    SELECT strftime(d.full_date, '%Y-%m') AS bulan,
           p.product_id,
           arg_max(p.nama_produk, p.valid_from) AS nama_produk_tercatat,
           sum(f.nilai_paid_rp) AS nilai_paid_rp
    FROM fact_sales_item f
    JOIN dim_date d ON d.date_sk = f.date_sk
    JOIN dim_outlet o ON o.outlet_sk = f.outlet_sk
```

```

JOIN dim_product p ON p.product_sk = f.product_sk
WHERE o.outlet_id = 'OUT-A'
      AND d.full_date >= DATE '2025-01-01' AND d.full_date < DATE '2025-07-01'
      AND f.status_transaksi = 'PAID'
GROUP BY 1, 2
), peringkat_produk AS (
  SELECT *, dense_rank() OVER (PARTITION BY bulan ORDER BY nilai_paid_rp DESC) AS peringkat
  FROM penjualan_produk
)
SELECT bulan, product_id, nama_produk_tercatat, nilai_paid_rp, peringkat
FROM peringkat_produk
WHERE peringkat <= 3
ORDER BY bulan, peringkat, product_id;

```

q03.sql

Sumber repo: [sql/40_analytics/q03.sql](#)

```

-- UTS K6/T2/k9 – Q03: Berapa transaksi REFUND dan rasionya tiap bulan?
-- Keluaran: bulan | transaksi_refund | transaksi_tercatat | rasio_refund_persen.
-- Grain keluaran: satu bulan WIB; denominator adalah ID transaksi valid yang memiliki item di fact.
-- Jawaban harapan: Juni 2025 memiliki 8 transaksi REFUND dari 420 transaksi tercatat, sekitar 1,90%.
-- Bukti: docs/profile_t2_k9_slice.md, bagian validasi analitik; ini rasio status, bukan nilai uang refund.
SELECT strftime(d.full_date, '%Y-%m') AS bulan,
       count(DISTINCT CASE WHEN f.status_transaksi = 'REFUND' THEN f.transaction_id END) AS transaksi_refund,
       count(DISTINCT f.transaction_id) AS transaksi_tercatat,
       round(100.0 * count(DISTINCT CASE WHEN f.status_transaksi = 'REFUND' THEN f.transaction_id END)
            / nullif(count(DISTINCT f.transaction_id), 0), 2) AS rasio_refund_persen
FROM fact_sales_item f
JOIN dim_date d ON d.date_sk = f.date_sk
JOIN dim_outlet o ON o.outlet_sk = f.outlet_sk
WHERE o.outlet_id = 'OUT-A'
      AND d.full_date >= DATE '2025-01-01' AND d.full_date < DATE '2025-07-01'
GROUP BY 1
ORDER BY 1;

```

5. Kamus metrik, gaming, dan guard

Metrik 1 — Nilai item PAID valid Outlet A

#	Field	Isi
1	Nama metrik	Nilai item PAID valid Outlet A (nilai_item_paid_rp)
2	Definisi (satu kalimat, tanpa jargon)	Jumlah nilai item pada transaksi Outlet A berstatus PAID yang lolos aturan kualitas selama periode yang dipilih, setelah diskon per item.
3	Rumus SQL	$\text{SUM}(f.\text{nilai_paid_rp})$ dengan $\text{nilai_paid_rp} = \text{qty_item} * \text{harga_satuan_rp} - \text{diskon_item_rp}$ untuk PAID valid dan 0 untuk REFUND/VOID; SQL lengkap: <code>sql/50_metrics/nilai_item_paid.sql</code> .
4	Grain	Dasar: satu item_id per transaksi; keluaran SQL: satu tanggal WIB per outlet; total enam bulan adalah SUM tanggal tersebut.
5	Tabel sumber	<code>fact_sales_item + dim_date + dim_outlet</code> ; <code>outlet_sk</code> fact berasal dari lookup <code>outlet_id</code> header <code>transactions.csv</code> , sedangkan nilai item berasal dari <code>transaction_items.csv</code> .
6	Owner (jabatan bernama)	Manajer Outlet A (usulan pemilik keputusan untuk disetujui kelompok).
7	Time basis	Tanggal transaksi lokal WIB (Asia/Jakarta), 1 Januari 2025 inklusif sampai 1 Juli 2025 eksklusif; string Z dikonversi dari UTC.
8	Satuan	Rupiah (Rp), dengan asumsi kerja bahwa <code>harga_satuan</code> dan <code>diskon</code> pada sumber adalah nominal rupiah.
9	Dimensi yang boleh dipotong	Tanggal/bulan, produk/kategori, dan pelanggan; outlet hanya OUT-A melalui <code>dim_outlet</code> sebagai batas cakupan. Setiap fact hanya cocok ke satu versi produk. Jangan SUM harga satuan atau rasio.
10	Filter default	<code>outlet_id='OUT-A'</code> , tanggal WIB [2025-01-01, 2025-07-01) dan item fact yang telah lolos validasi; hanya status PAID menyumbang nilai.
11	Arti nilai kosong	Jika tidak ada baris fact untuk tanggal/outlet, hasil tidak tersedia/NULL (tidak ada data); jika ada baris valid tetapi semuanya VOID/REFUND, hasil 0. Ini bukan bukti outlet tutup.

#	Field	Isi
12	Versi	v1, berlaku mulai 28 September 2026 untuk desain UTS; perubahan rumus/status harus menaikkan versi.

Cara membaca. Ini adalah nilai penjualan item PAID *tercatat* dengan aturan kualitas yang dinyatakan, bukan laba dan bukan nilai penjualan bersih setelah refund. Data REFUND memiliki qty positif dan negatif tanpa tautan ke transaksi asal; penyesuaian net setelah refund memerlukan aturan tambahan. Untuk sumber k9, profiling baca-saja dengan kebijakan karantina dalam desain memberi perkiraan Rp181.778.000 selama enam bulan. Angka ini bukan hasil menjalankan fact atau loader.

Asumsi yang harus diputuskan pemilik data. Diskon diasumsikan rupiah nominal per item karena nilai teramati 0/1.000/2.000/5.000 dan tidak melebihi subtotal item positif; CSV tidak memuat metadata satuannya. Sebelum memakai metrik untuk pelaporan keuangan resmi, konfirmasi satuan diskon dan alasan selisih total_bayar header terhadap jumlah item. Jika asumsi salah, revisi rumus dan versi metrik.

Satu cara gaming

Pada proses load, diskon item PAID dapat diam-diam diubah menjadi nol. Penjualan terlihat lebih besar meskipun jumlah/harga item tidak berubah.

Guard test yang menangkap gaming

Bandingkan diskon setiap item PAID yang lolos staging dengan nilai yang tersimpan di fact; pelanggaran harus **0** setelah implementasi. Query ini adalah **rancangan guard untuk warehouse masa depan**, bukan salah satu enam test sumber yang memiliki perkiraan_baris UTS. stg_item_valid berasal dari alur pada pipeline/DESIGN_load.md; implementasi harus menyediakan view/table audit yang setara jika staging bersifat sementara.

```
SELECT count(*) AS pelanggaran
FROM stg_item_valid s
JOIN stg_transaction_k9 t ON t.transaction_id=s.transaction_id
LEFT JOIN fact_sales_item f ON f.item_id=s.item_id
WHERE upper(trim(t.status))='PAID'
  AND (f.item_id IS NULL
        OR f.diskon_item_rp IS DISTINCT FROM TRY_CAST(s.diskon AS DECIMAL(18,2))
        OR f.nilai_paid_rp IS DISTINCT FROM
            TRY_CAST(s.qty AS DECIMAL(18,2))*TRY_CAST(s.harga_satuan AS DECIMAL(18,2))
            - TRY_CAST(s.diskon AS DECIMAL(18,2)));
```

LEFT JOIN dalam guard juga menangkap item PAID valid yang hilang seluruhnya dari fact. stg_item_valid sudah mengecualikan karantina yang sah berdasarkan reason code, sehingga perbandingan memakai populasi yang sama.

Sumber repo: sql/50_metrics/nilai_item_paid.sql

```
-- UTS K6/T2/k9 – Definisi eksekutabel metrik pada docs/kamus_metrik.md.
-- Grain keluaran: hari kalender WIB, OUT-A. Fact hanya memuat item transaksi valid.
-- REFUND/VOID tidak menambah nilai_paid_rp; metrik ini bukan laba dan bukan net setelah refund.
SELECT d.full_date AS tanggal_wib,
       o.outlet_id,
       sum(f.nilai_paid_rp) AS nilai_item_paid_rp,
       count(DISTINCT CASE WHEN f.status_transaksi='PAID' THEN f.transaction_id END) AS transaksi_paid
```

```
FROM fact_sales_item f
JOIN dim_date d ON d.date_sk=f.date_sk
JOIN dim_outlet o ON o.outlet_sk=f.outlet_sk
WHERE o.outlet_id='OUT-A'
      AND d.full_date>=DATE '2025-01-01' AND d.full_date<DATE '2025-07-01'
GROUP BY 1,2
ORDER BY 1;
```

6. Batas desain

Keputusan	Bukti/alasan	Alternatif yang ditolak dan akibatnya
Gunakan item_id sebagai grain	ID item tersedia; 42 duplikat pada slice identik setelah diperiksa.	(transaction_id, product_id) dapat menggabungkan item produk yang sama dalam satu transaksi.
Batasi k9 setelah konversi WIB	Penugasan Outlet A/enam bulan; 375 header slice bertanda Z.	Filter string mentah/UTC dapat memindah transaksi dekat batas hari/periode.
Karantina empat ID header konflik	Satu ID memiliki dua payload berbeda.	Memilih nilai pertama/terakhir tanpa aturan akan mengubah tanggal, pelanggan, atau nilai.
Nilai item PAID valid sebagai metrik	REFUND memuat qty positif dan negatif tanpa link asal.	Mengurangkan seluruh REFUND sebagai angka bersih akan mengklaim refund yang belum terukur.
Produk Type 2 dengan seed "as known"	Hanya satu snapshot master, sementara 110 dari 120 tanggal harga mulai setelah 1 Januari 2025.	Memakai harga master saat ini sebagai harga historis akan mengubah nilai transaksi lampau.
Gunakan tiga dimensi domain dan kalender	Koreksi dosen menegaskan tiga domain + dim_date; master outlet tersedia dan OUT-A perlu divalidasi.	Menyimpan outlet hanya sebagai atribut fact menghilangkan FK outlet dan tidak mengikuti jumlah dimensi terbaru.
Full scan enam bulan setiap run	Sumber tidak punya updated_at yang andal.	Watermark hanya bulan terbaru dapat melewati koreksi lama.

Pertanyaan yang belum dapat dijawab: Produk mana yang menghasilkan *laba bersih* terbesar di Outlet A? CSV menyediakan qty, harga jual, diskon, dan pembayaran header, tetapi tidak menyediakan HPP historis maupun biaya operasional/alokasinya. Untuk menjawab diperlukan data biaya/HPP menurut periode, aturan alokasi biaya, dan definisi laba. Data tambahan tersebut hanya dicatat sebagai kebutuhan masa depan; UTS tidak memakai dataset lain.

Status: ini adalah rancangan UTS. Pertanyaan laba bersih pada bagian ini belum dapat dijawab dari dataset yang diberikan.